

Rapport de Projet de 2ème année OratoChrono (Software)

Par

Tristan BRAU – Mehdi BOULHARTS – Anas HSAINI

Sous la supervision de

M. Antoine TAUVEL



Ecole nationale supérieure de l'électronique et de ses applications

CERGY – 95000, FRANCE

Avril 2025

Introduction

Objectif du travail :

Ce projet s'inscrit dans le cadre du développement d'un outil numérique nommé Orato Chrono, visant à améliorer l'organisation et la gestion du temps de parole lors des réunions entre élèves et enseignants de l'ENSEA. En tant qu'équipe software, notre objectif principal est de concevoir un site internet sécurisé, accessible uniquement aux membres de l'école via l'authentification CAS (Central Authentication Service). Ce site permettra la création, la planification et le suivi des réunions.

Ce projet est réalisé dans le cadre d'un travail en équipe qui s'inscrit dans le cursus de l'ENSEA, combinant des compétences en développement web, gestion de bases de données et intégration de services externes. L'équipe hardware travaille en parallèle sur un dispositif physique (le "chrono") destiné à matérialiser et à réguler le temps de parole de chaque intervenant pendant les réunions.

Dans ce compte rendu, nous détaillerons :

- 1- La conception du site en HTML, CSS et JavaScript,
- 2- L'intégration de Firebase (base de données NoSQL) pour stocker les informations relatives aux réunions,
- 3- Le fonctionnement de l'authentification CAS pour restreindre l'accès aux membres de l'ENSEA,

1- La conception du site en HTML, CSS et JavaScript,

Au départ, notre objectif était de concevoir le site web du projet Orato Chrono en Java, en raison de sa réputation de langage robuste et polyvalent. Cependant, aucun membre de notre équipe ne maîtrisait Java, ce qui nous a contraints à en entamer l'apprentissage dès les premières semaines. Rapidement, en commençant à développer, nous avons constaté que réaliser un site web complet en Java allait être beaucoup trop complexe, notamment en raison de la mise en place nécessaire d'un framework comme Spring, ainsi que de la gestion de l'interface utilisateur.

Face à cette difficulté, deux choix s'offraient à nous :

- 1- Développer une application mobile en React Native, un framework que nous ne connaissions pas non plus,
- 2- Ou bien nous orienter vers une stack web plus accessible, à savoir HTML, CSS et JavaScript, que nous avons décidé d'apprendre de zéro.

Après quelques recherches, de nombreux retours d'expérience sur Internet conseillaient fortement de commencer par HTML, CSS et JavaScript pour tout projet web débutant. Nous avons donc choisi cette voie. Pour apprendre ces technologies, nous avons utilisé :

- Les tutoriels d'OpenClassrooms,
- Des vidéos explicatives sur YouTube,
- Les enseignants à l'ENSEA.

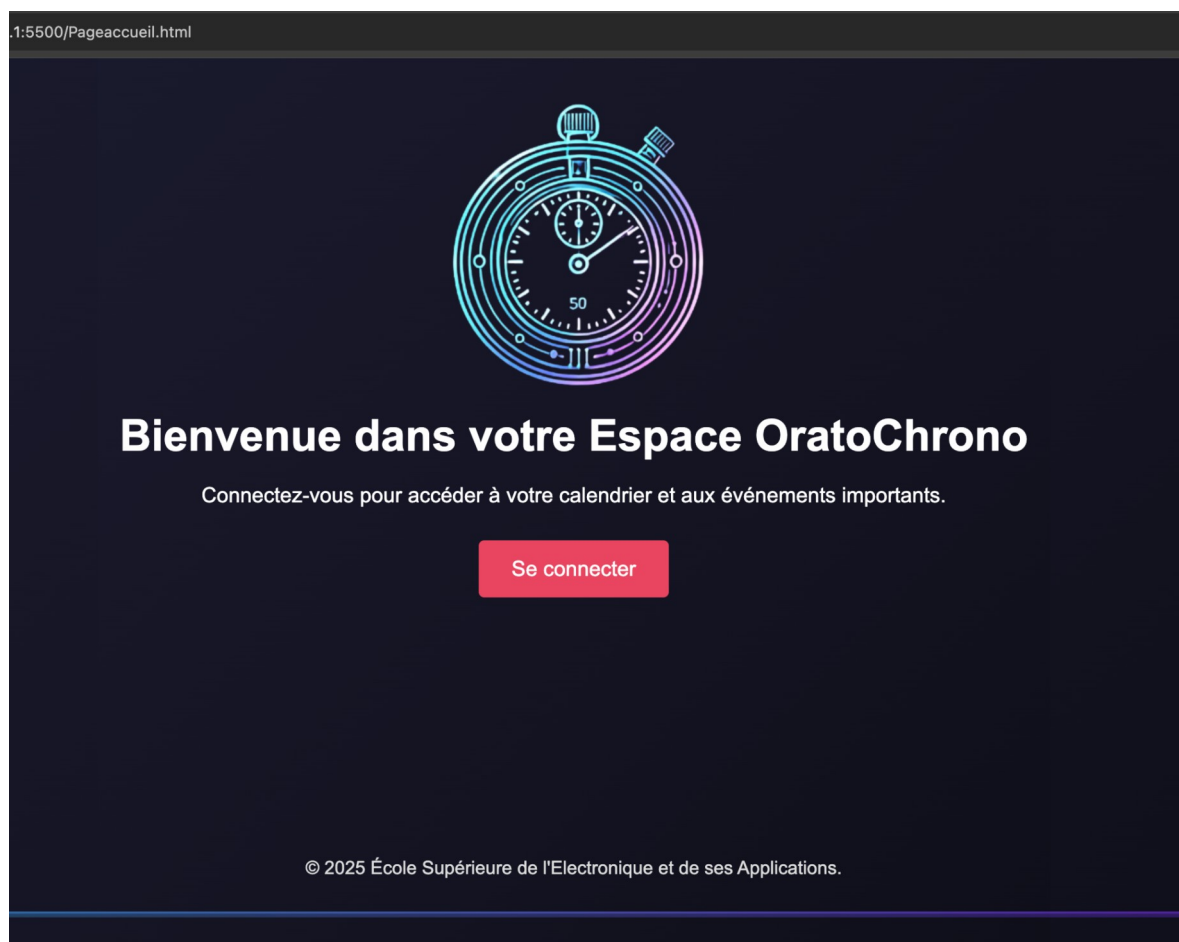
Ce choix s'est révélé particulièrement judicieux pour plusieurs raisons, tant sur le plan pédagogique que technique.

- HTML (HyperText Markup Language) constitue la structure de base du site. Il nous permet d'organiser logiquement le contenu de chaque page : titres, paragraphes, formulaires de création de réunion, boutons d'action (comme "Lancer une réunion"), menus de navigation, etc. HTML est essentiel pour garantir une navigation claire et cohérente. Sans cette base, aucune interface utilisateur ne pourrait exister.
- CSS (Cascading Style Sheets) apporte une couche de présentation et de style à ce contenu brut. Grâce au CSS, nous avons pu personnaliser l'apparence du site : choix des couleurs, polices, marges, tailles, animations légères, mise en page responsive... Cela nous a permis de proposer une interface plus moderne, ergonomique et agréable à utiliser, ce qui est indispensable pour un outil destiné à l'ensemble des élèves et enseignants de l'ENSEA.

- JavaScript, quant à lui, est le langage de la logique. Il nous a permis d'ajouter de l'interactivité : vérification des formulaires, affichage dynamique des réunions, mises à jour sans recharger la page, gestion du compte utilisateur, interactions avec la base de données, etc. C'est avec JavaScript que nous avons pu rendre notre site fonctionnel et intelligent, notamment en assurant la communication avec Firebase (lecture et écriture des réunions) et l'intégration de l'authentification CAS.

Par rapport aux autres solutions envisagées, cette approche a été plus rapide à mettre en œuvre, mieux documentée en ligne, et nous a permis de comprendre en profondeur le fonctionnement d'un site web. Elle offre également une excellente compatibilité avec Firebase, notre base de données, et avec l'intégration du système de connexion CAS de l'ENSEA.

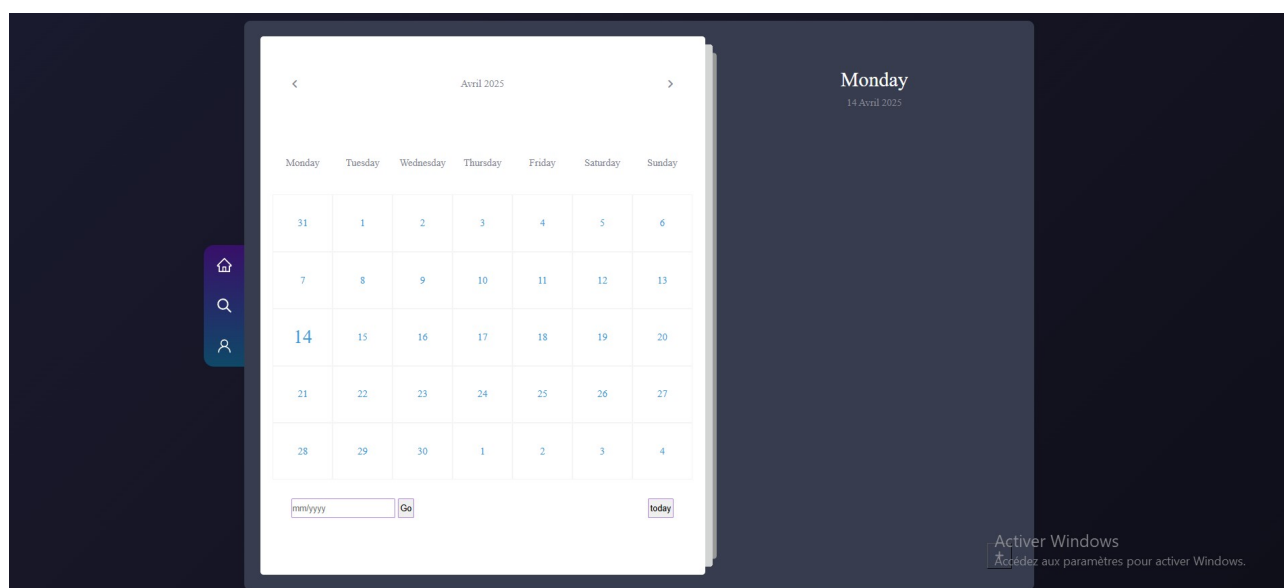
Actuellement, le site tourne en local, c'est-à-dire qu'il est exécuté sur nos propres machines via un simple serveur local. Cela nous permet de tester les fonctionnalités en cours de développement, de corriger les bugs et d'itérer rapidement avant une mise en ligne future. Cette méthode est particulièrement adaptée dans une phase de prototypage, car elle offre une grande souplesse et ne nécessite pas d'hébergement immédiat. Pour avoir l'avancement et les erreurs de notre site il suffit de faire cliquer droit et de cliquer sur inspecter.



La page présentée sur la capture d'écran ci-dessous a été entièrement réalisée en HTML et CSS, sans utilisation de framework externe. Toute la structure (titres, blocs, zones de texte, menu, etc.) ainsi que la mise en forme (couleurs, marges, alignements, tailles de police, styles des éléments) ont été conçues manuellement via des feuilles de style CSS.

À ce stade, aucune logique JavaScript complexe n'est intégrée sur la page, à l'exception du bouton, qui possède un script JS simple pour gérer une action basique (par exemple : affichage, redirection ou lancement d'une fonction). Cela nous a permis de nous concentrer dans un premier temps sur le design statique et l'ergonomie, avant d'intégrer progressivement la logique dynamique.

Ce travail nous a permis de bien maîtriser les bases de la construction d'interfaces web, tout en posant les fondations visuelles du site Orato Chrono.



Première Page

La page affichée ci-dessous correspond à l'interface principale du calendrier du site Orato Chrono. Elle a été conçue en HTML et CSS, avec une touche de JavaScript uniquement pour certaines interactions, notamment les boutons. Elle permet d'afficher les réunions et d'interagir avec celles-ci de manière intuitive.

Voici les fonctionnalités intégrées à cette page :

- Affichage des réunions : les événements créés sont visibles directement sur le calendrier.
- Démarrage d'une réunion : en cliquant sur un événement dans la colonne de droite, il est possible de lancer la réunion.
- Bouton "Maison" : renvoie vers la page d'accueil du site.
- Bouton "Personne" : redirige vers le site de l'école.
- Barre de recherche et loupe : permet de rechercher un événement par son nom. Si une correspondance est trouvée, le jour de l'événement s'affiche en rouge dans le calendrier pour le repérer rapidement.

- Bouton "+" : ouvre un formulaire pour ajouter un nouvel événement.
- Les jours contenant des événements sont signalés par une barre violette en dessous du numéro du jour.
- L'utilisateur peut naviguer dans les mois soit en cliquant sur les flèches gauche/droite, soit en choisissant un mois précis dans la barre de recherche.
- Le bouton "Today" permet de revenir instantanément au mois en cours.

Important : ce compte rendu est rédigé à la mi-avril, or nous rencontrons actuellement un problème d'accès à Firebase (probablement lié à des restrictions de permissions ou des modifications de configuration). À cause de cela, certaines fonctionnalités dynamiques ne sont pas visibles sur la capture, notamment l'affichage réel des événements stockés dans la base de données et le lancement des réunions. Toutefois, ces fonctions sont bien codées et fonctionnaient correctement avant la perte d'accès à Firebase.

The screenshot shows a mobile application interface for adding a new conference. The form is displayed on a dark background. It includes the following fields and options:

- Nom de la conférence :** A text input field with a placeholder "Entrez le nom de la confère".
- Date :** A date picker showing "jj/mm/aaaa".
- Heure de début :** A time picker showing "--:--".
- Salle :** A text input field with a placeholder "Ex: A102".
- Administrateur :** A text input field with a placeholder "admin@example.com" and a user icon.
- Intervenants (séparés par des virgules) :** A text input field with a placeholder "Ex: Alice Dupont, Bob Mart".
- Déroulé :** A text input field with a placeholder "Ex: 5min Introduction, 3min".
- Temps par intervenant (format: Nom=Temps, Nom=Temps) :** A text input field with a placeholder "Ex: Alice Dupont=10, Bob M".
- Ajouter la conférence** : A button at the bottom of the form.

In the bottom right corner of the screen, there is a watermark that says "Activer Windows" and "Accédez aux paramètres pour activer Windows."

Page info

La page présentée ci-dessous permet de créer de nouvelles réunions dans le cadre du projet Orato Chrono. Elle a été développée en HTML et CSS, avec une touche de JavaScript pour la gestion du formulaire et l'envoi des données vers Firebase.

Cette interface regroupe tous les champs nécessaires pour configurer une réunion complète.

L'utilisateur doit renseigner les éléments suivants :

- Nom de la conférence : titre ou thème principal.
- Date de la réunion (avec un sélecteur de date).
- Heure de début (avec un sélecteur d'heure).
- Salle dans laquelle aura lieu la réunion (ex. A102).
- Administrateur : généralement l'organisateur ou responsable de la conférence.

- Intervenants : une liste de participants, séparés par des virgules.
- Déroulé : description rapide du programme ou du plan de la réunion.
- Temps par intervenant : durée de parole allouée à chaque participant, indiquée sous la forme Nom=Temps, séparés par des virgules.

Une fois le formulaire complété, l'utilisateur clique sur "Ajouter la conférence" pour enregistrer la réunion.

2- L'intégration de Firebase (base de données NoSQL) pour stocker les informations relatives aux réunions.

Pour stocker toutes les informations relatives aux réunions (nom, date, intervenants, durée, etc.), nous avons utilisé Firebase, un service de base de données NoSQL en temps réel proposé par Google.

Nous avons intégré Firebase à notre site à l'aide de l'API officielle. Cette API nous permet d'envoyer, récupérer et modifier les données directement depuis le code JavaScript de notre application web.

Le morceau de code ci-dessous représente le début de l'intégration de Firebase. Il permet :

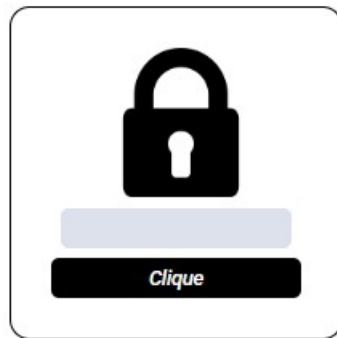
- d'initialiser l'application Firebase avec les paramètres de notre projet (ID, URL de la base de données),
- d'établir une connexion avec la base de données,
- de définir des références précises dans la base (par exemple ici, une référence nommée "Conference" où seront stockées les conférences créées depuis le formulaire du site).

Ce code est indispensable pour faire fonctionner correctement les interactions entre le site web et la base Firebase.

```
import {initializeApp} from "https://www.gstatic.com/firebasejs/9.15.0/firebase-app.js"
import {getDatabase,ref,update,push,set} from "https://www.gstatic.com/firebasejs/9.15.0/firebase-database.js"

const appSettings = {  projectId: "Playground", databaseURL : "https://enseatimekeeper-default-rtdb.europe-west1.firebaseio.com/" }
const app = initializeApp(appSettings) //sert à initialiser et configurer votre application Firebase avec les paramètres que vous fournissez. El
console.log(app)
const database = getDatabase(app)//Cette fonction permet d'accéder à votre base de données Firebase.
console.log(database)
// const userId= ref(database, "userId")
const Conference= ref(database, "Conference")// ref sert a spécifier un chemin spécifique dans ta base de donnée

const inputField = document.getElementById("input-field")
const addButtonEl = document.getElementById("add-button")
const Image =document.getElementById("image1")
```



Voici la première page que nous avons créée en utilisant les API de Firebase. Elle permet de modifier une valeur déjà présente dans la section "Conference" de la base de données.

L'utilisateur peut écrire une nouvelle valeur dans la barre de saisie, et en validant, celle-ci remplace l'ancienne valeur stockée dans la base.

Cette première version simple nous a permis de tester la communication entre notre site et Firebase, et de comprendre le fonctionnement des opérations de mise à jour dans une base NoSQL.

Après cette première expérimentation réussie avec Firebase, nous avons progressivement intégré les fonctionnalités de la base de données dans les autres pages du site, en particulier dans la page principale du calendrier, présentée dans la partie 1 du compte rendu.

Sur cette page, Firebase est utilisée pour :

- 1- récupérer les réunions enregistrées dans la base et les afficher dynamiquement dans le calendrier,
- 2- ajouter une nouvelle réunion via le formulaire de création,
- 3- lancer une réunion en cliquant sur un événement, ce qui permet à la partie hardware de synchroniser le temps de parole.

Chaque interaction est donc directement liée à Firebase, ce qui rend le site entièrement fonctionnel en temps réel, sans avoir besoin de recharger la page ou d'utiliser un serveur classique.

Malheureusement, en raison d'un problème d'accès à Firebase au moment de la rédaction (mi-avril), certaines de ces fonctionnalités ne sont pas visibles sur les captures d'écran actuelles, mais elles ont bien été développées et testées avec succès en amont.

3- Le fonctionnement de l'authentification CAS pour restreindre l'accès aux membres de l'ENSEA,

```
// URL de base du serveur CAS et des pages accueils et
const CAS_BASE_URL = 'https://identites.ensea.fr/cas';
const SERVICE_URL = encodeURIComponent('http://127.0.0.1:5500/Pageaccueil.html');
const SERVICE_URL2 = encodeURIComponent('http://127.0.0.1:5500/index.html');
const SERVICE_URL3 = encodeURIComponent('http://127.0.0.1:5500/premierepage.html');

// Écouteur d'événement pour le bouton "Se connecter"
const CAS_Button = document.getElementById("connect-button")
CAS_Button.addEventListener("click", function() { // a la place de function() on peut mettre ()=>
  // Redirection vers le serveur CAS avec le paramètre 'service'
  console.log("CAS connected")
  window.location.href = `${CAS_BASE_URL}/login?service=${SERVICE_URL3}`;
});
```

Entre la page d'accueil de notre site et la première page fonctionnelle (celle où les réunions sont créées ou consultées), nous avons voulu mettre en place une authentification sécurisée afin de limiter l'accès uniquement aux élèves et enseignants de l'ENSEA. Pour cela, nous avons utilisé le système CAS (Central Authentication Service) propre à l'école.

Qu'est-ce que le CAS ?

Le CAS (Central Authentication Service) est un système d'authentification unique utilisé par de nombreuses institutions académiques. Il permet aux utilisateurs (élèves, professeurs, personnel) de se connecter une seule fois via leur compte institutionnel pour accéder à plusieurs services (Moodle, etc.), sans avoir à ressaisir leur mot de passe à chaque fois.

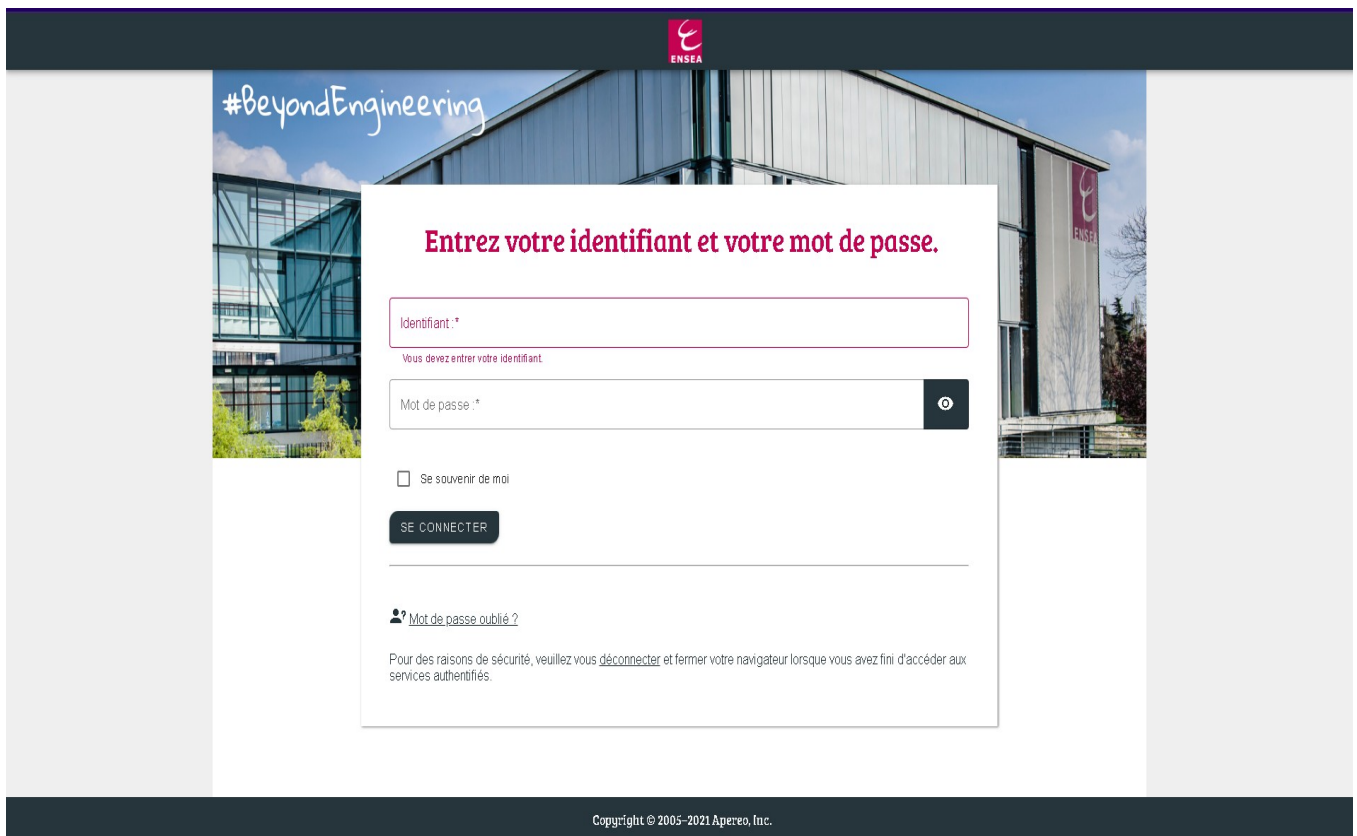
Comment fonctionne l'intégration CAS dans notre projet ?

Voici le fonctionnement général que nous avons intégré (ou prévu d'intégrer) :

1. Lorsqu'un utilisateur tente d'accéder au site, il est redirigé vers la page de connexion CAS de l'ENSEA.
2. Il s'authentifie avec ses identifiants ENSEA (ceux utilisés pour l'ENT).
3. Une fois connecté, le CAS génère un ticket de session temporaire qui est vérifié par notre site.
4. Si le ticket est valide, l'utilisateur est autorisé à accéder aux pages du site ; sinon, il reste bloqué.

Cette méthode nous permet de :

1. Garantir que seuls les membres de l'ENSEA accèdent au site,
2. Éviter d'avoir à gérer des mots de passe manuellement (ce qui serait peu sécurisé),
3. Profiter d'une solution éprouvée et institutionnelle déjà en place dans l'école.



Conclusion

Ce projet nous a permis de mener à bien une réalisation logicielle complète, depuis l'idée initiale jusqu'au déploiement d'une première version fonctionnelle. En tant qu'équipe software du projet Orato Chrono, notre objectif était de créer un site web permettant de gérer les réunions entre élèves et enseignants de l'ENSEA, avec une attention particulière portée à la gestion du temps de parole.

Nous avons rencontré plusieurs défis techniques, notamment :

- le changement de langage (du Java vers HTML/CSS/JavaScript) pour mieux s'adapter à nos compétences et aux outils disponibles,
- l'apprentissage et l'intégration de Firebase, pour stocker les réunions et rendre le site interactif,
- la volonté d'intégrer une authentification sécurisée CAS, afin de restreindre l'accès aux membres de l'école uniquement.

Au fil du projet, nous avons su monter en compétence, apprendre de nouvelles technologies (web, base de données NoSQL, API Firebase), et concevoir une interface claire, interactive et évolutive. Le site fonctionne actuellement en local, mais il est conçu pour être déployé sur un serveur sécurisé par la suite.

Enfin, ce projet s'inscrit dans une logique plus large de collaboration avec l'équipe hardware, en lien avec l'objectif final d'optimiser les réunions en mesurant et gérant le temps de parole grâce à un chrono physique synchronisé avec le site.